hashlock.

Security Audit

Predmart (DeFi)

April 2026

info@hashlock.com.au

Hashlock Pty Ltd

Table of Contents

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

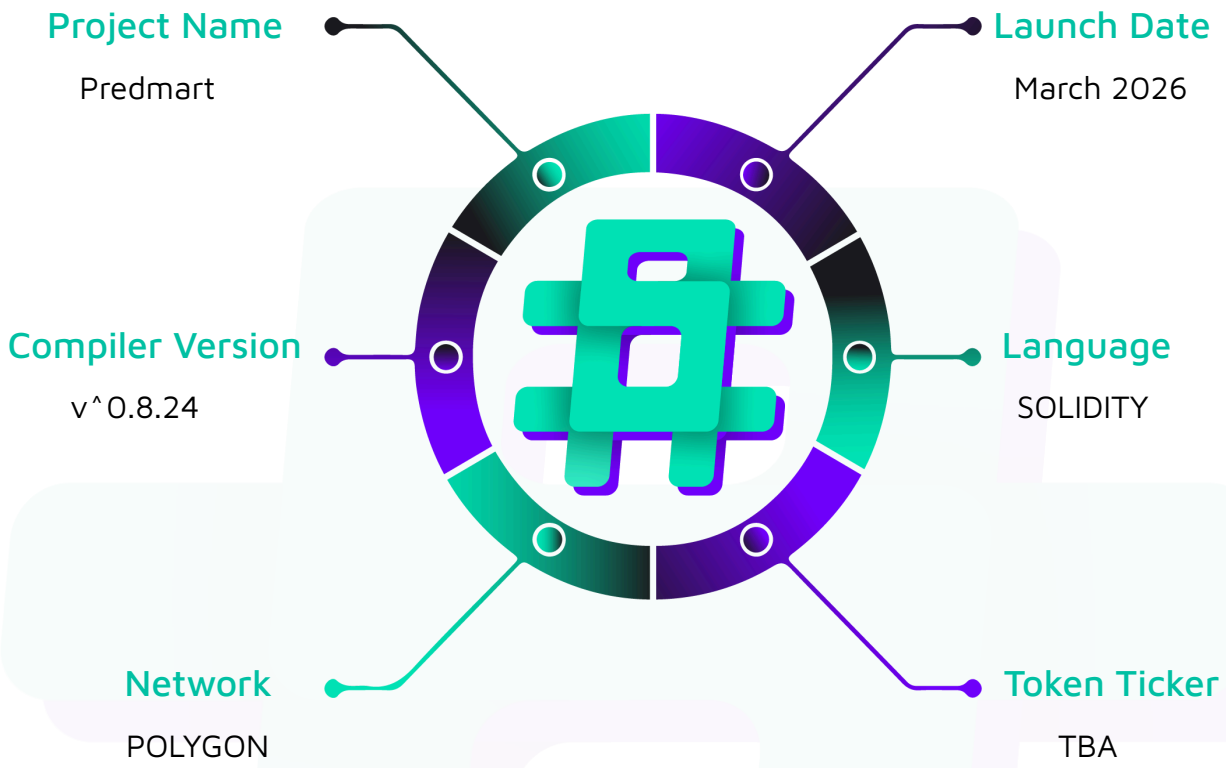
The Predmart team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

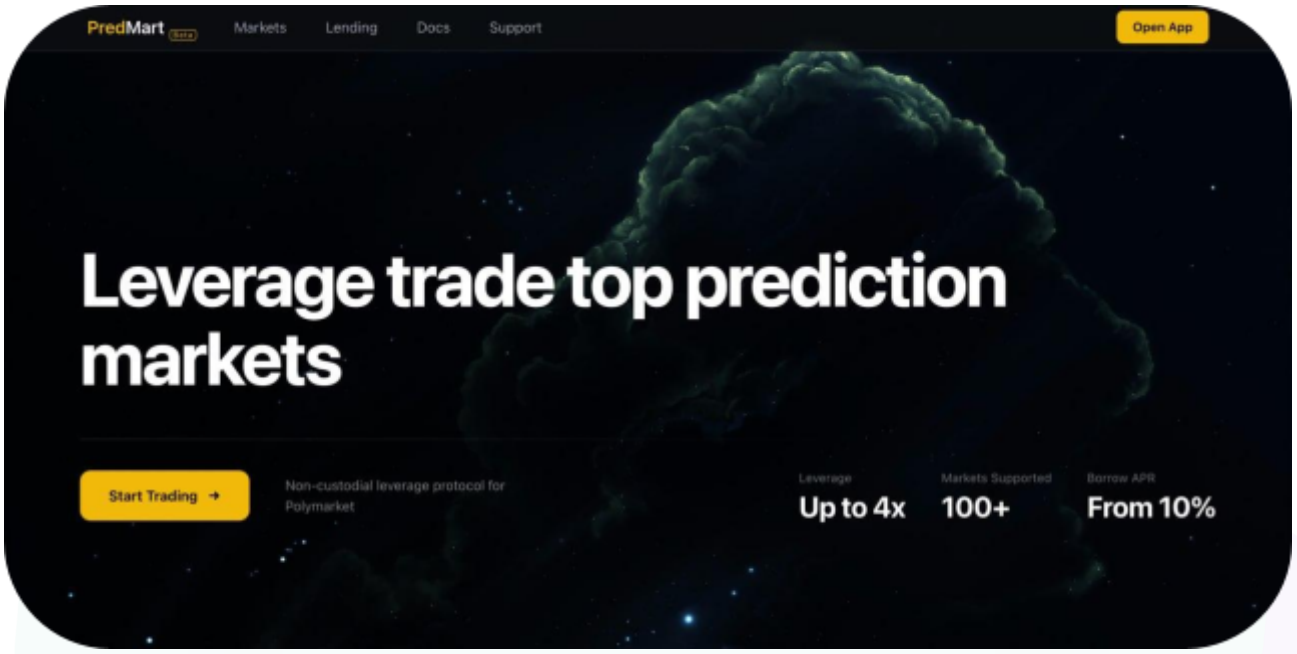
PredMart is a non-custodial DeFi lending protocol built for the Polymarket prediction market ecosystem. Users deposit their Polymarket outcome shares (ERC-1155 CTF tokens) as collateral and borrow USDC against them, unlocking liquidity from positions that would otherwise be locked until market resolution. The borrowed USDC enables leveraged trading on Polymarket - borrowers can amplify their exposure up to ~4x - or general capital efficiency. On the other side, lenders supply USDC to an ERC-4626 vault and earn yield generated from borrower interest. PredMart does not operate its own prediction markets; it is the credit and leverage layer for users trading on Polymarket. Deployed on Polygon mainnet since March 2026.

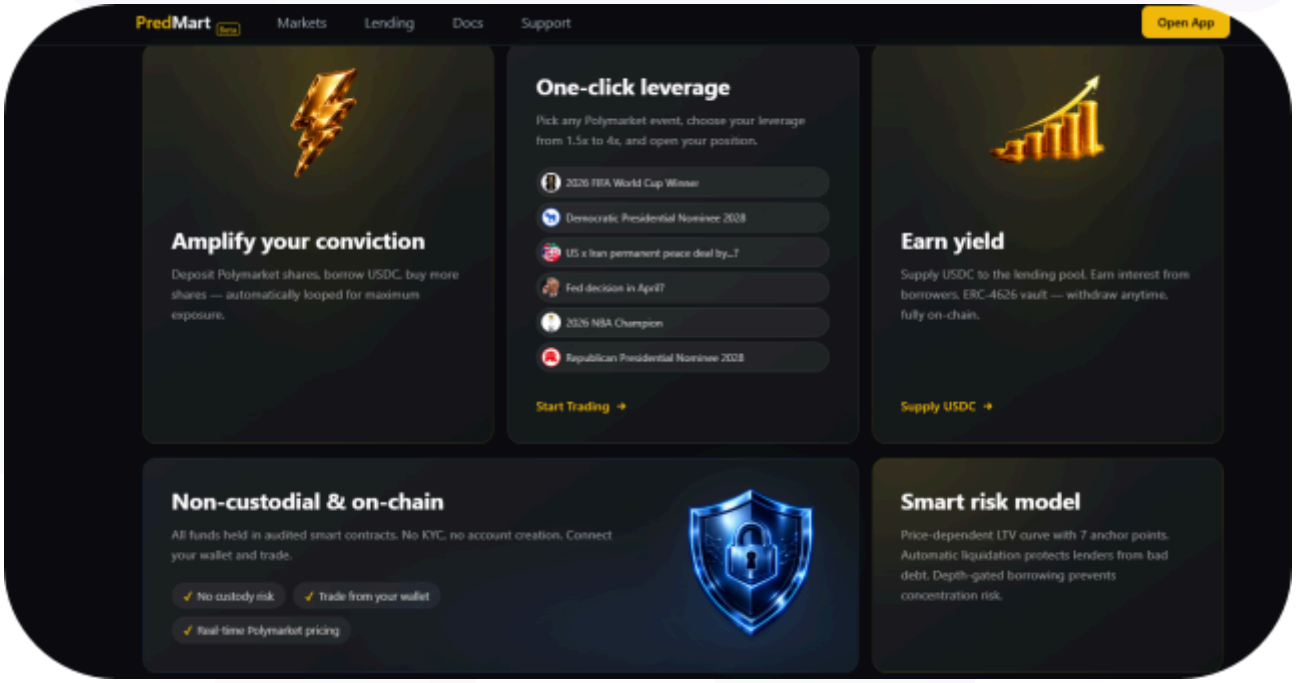
Project Name: Predmart
Project Type: Defi
Compiler Version: ^0.8.24
Website: https://predmart.com/
Logo:

Visualised Context:



Project Visuals:





Audit Scope

We at Hashlock audited the solidity code within the Predmart project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	**Predmart Smart Contracts**
Network	**Polygon**
Language	**Solidity**
Audit Date	**April, 2026**
Contract 1	PredmartLendingPool.sol
Contract 2	PredmartPoolExtension.sol
Contract 3	PredmartPoolLib.sol
Contract 4	PredmartOracle.sol
Contract 5	PredmartTypes.sol
Contract 6	PredmartLeverageModule.sol
Audited GitHub Commit Hash	99ab69da4e4c67dc21992d341116e79ba42937ea
Fix Review GitHub Commit Hash	11502df55e0dba60b253492b79d75fd1bd97a8dd

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the Audit Findings section. The list of audited assets is presented in the Audit Scope section and the project's contract functionality is presented in the Intended Smart Contract Functions section.

All vulnerabilities initially identified have now been resolved.

Hashlock found:

3 High severity vulnerabilities

2 Medium severity vulnerabilities

6 Low severity vulnerabilities

2 QA

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour

PredmartLendingPool.sol
Main upgradeable lending pool ERC-4626 USDC vault for lenders + borrow against Polymarket CTF ERC-1155 shares for borrowers, with meta-tx relayer + extension delegatecall pattern.
- Allows lenders to:
  - Deposit/mint USDC into the ERC-4626 vault and withdraw/redeem subject to available liquidity
- Allows borrowers to:
  - Deposit CTF shares as collateral
  - Borrow USDC via trusted relayer
  - Repay debt
  - Withdraw collateral via trusted relayer
  - Run leverage loops via relayer
  - Initiate a pool funded flash close of the whole position via relayer
- Allows the relayer to:
  - Execute borrower actions with the signed authorizations
  - Execute liquidations
- Allows the admin to:
  - Authorize UUPS upgrades and set the extension address
	Contract achieves this functionality.

PredmartPoolExtension.sol Extension contract called via delegatecall from PredmartLendingPool fallback; contains admin/governance + settlement flows.	**Contract achieves this functionality.**

- Allows the admin to:
  - Set pause state and freeze/unfreeze specific tokenIds
  - Set per-token borrow cap
  - update timelock delay
  - Set operation fee, withdraw USDC operation fees, withdraw CTF fee shares, sweep orphaned redemption proceeds
  - Propose/execute/cancel timelocked changes
  - Withdraw protocol reserves and Expire a stuck leverage
- Allows anyone to:
  - Resolve a market with oracle-signed resolution
  - Close positions for lost markets
  - Redeem winning collateral from CTF into USDC
  - Settle a borrower after redemption
  - Expire a timed-out pending flash-close
- Allows the relayer to:
  - Settle a pending flash-close after offchain CLOB sale	

PredmartOracle.sol Signature-verification library for oracle-signed price and resolution messages.	**Contract achieves this functionality.**
PredmartPoolLib.sol Pure math library (external funcs) extracted to keep the main pool under EIP-170 size including liquidation, health factor, interest and risk model calculations.	**Contract achieves this functionality.**

PredmartLeverageModule.sol - Safe Module that authorizes leverage operations on the borrower's Gnosis - Safe after verifying an EIP-712 signature from a Safe owner. Provides the Safe-side consent required by PredmartLendingPool's leverage flow withoutper-trade Safe transactions from the user.	

Code Quality

This audit scope involves the smart contracts of the Predmart project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring were recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Predmart project smart contract code in the form of GitHub access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.
Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-severity issues should be resolved before deployment. While they do not typically lead to a complete loss of funds, they may result in partial loss of funds or unintended behavior under certain conditions.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

High

[H-01] PredmartLendingPool#depositCollateralFrom - Single Safe owner can drain collateral from a shared Gnosis Safe

Description

`depositCollateralFrom` is intended to let a Gnosis Safe deposit its CTF shares as collateral. The caller authorization check accepts any individual Safe owner, but Gnosis Safe owners have no unilateral authority — only a threshold-satisfying multi-sig execution does. Any single owner can therefore pull all CTF shares out of the Safe and credit them to their own address.

Vulnerability Details

When `msg.sender != from`, `depositCollateralFrom` checks whether `msg.sender` is a registered owner of the Safe at `from` by calling `isOwner(address)`:

```
(bool ok, bytes memory data) = from.staticcall(
    abi.encodeWithSignature("isOwner(address)", msg.sender)
);
if (!ok || data.length < 32 || !abi.decode(data, (bool))) revert NotProxyOwner();

_depositCollateral(msg.sender, from, tokenId, amount);
```

`isOwner` on a Gnosis Safe returns `true` for any registered owner regardless of the configured signing threshold. Passing this check does not mean the Safe has authorized the action — it only means the caller appears in the Safe's owner list.

Once the check passes, `_depositCollateral` is called with `creditTo = msg.sender` and `from = safeAddress`. The CTF shares are transferred out of the Safe via the `setApprovalForAll` the Safe grants the LendingPool during onboarding, and the collateral position is recorded under the individual owner's address, not the Safe's.

Impact

Any one of a Safe's owners gains unilateral control over assets that are collectively owned, constituting a complete bypass of the Safe's access-control model and a full loss of funds for the remaining signers.

Recommendation

To support Safe and any other contract accounts, require a valid EIP-1271 signature from the Safe (`from.isValidSignature(hash, signature)`) constructed over a commitment to `(tokenId, amount, nonce, deadline)`, so the full threshold of owners must sign before any assets move.

Status

Resolved

[H-02] PredmartPoolExtension#pullUsdcForLeverage - Borrower can drain any address that has approved the contract

Description

`pullUsdcForLeverage` pulls USDC from `auth.allowedFrom` using only a signature from `auth.borrower`. Because `allowedFrom` is an unconstrained field in the borrower-signed struct, any attacker can set it to any victim who holds an outstanding ERC-20 approval to the contract, causing the contract to transfer the victim's USDC to the relayer.

Vulnerability Details

The `pullUsdcForLeverage` function transfers USDC with `safeTransferFrom(auth.allowedFrom, msg.sender, userAmount)`. The only authorization check is an EIP-712 signature from `auth.borrower`.

`auth.allowedFrom` appears inside the signed struct and is bound to the borrower's signature, but the victim at that address never provides any authorization. There is no requirement that `allowedFrom == borrower`, and no signature from `allowedFrom` is required.

An attacker constructs a `LeverageAuth` with `auth.allowedFrom` set to a victim's address, signs it as `auth.borrower`, and submits it to the relayer. The signature check passes and the relayer calls `pullUsdcForLeverage`, transferring the victim's USDC to the relayer address.

Any address that has approved the contract is at risk — including users who called `approve` before a legitimate `deposit()` call.

Impact

Any attacker can drain the full approved USDC balance from any address that has granted an ERC-20 allowance to the contract.

Recommendation

Require that `auth.allowedFrom == auth.borrower` so users can only authorize pulls from their own address or, if is required pulling from a third-party address like a gnosis safe, require a separate EIP-712 signature from `auth.allowedFrom` that commits to the specific `borrower`, `userAmount`, `tokenId`, `nonce` and `deadline` and validate it using OpenZeppelin `SignatureChecker.isValidSignatureNow()` to support both ECDSA and EIP-1271 signature checks before executing the transfer.

Status

Resolved

[H-03] PredmartLendingPool#leverageDeposit - Borrower can drain CTF shares from any address that has approved the contract

Description

`leverageDeposit` pulls CTF shares from `from` and credits them as collateral to `auth.borrower`'s position using only a signature from `auth.borrower`. Because `auth.allowedFrom` which determines the valid values for `from` is an unconstrained field in the borrower-signed struct, an attacker can set it to any victim who has granted `setApprovalForAll` to the contract, transferring the victim's shares into the attacker's collateral position.

Vulnerability Details

The function validates `from` against the signer-controlled `auth.allowedFrom`:

```
if (from != auth.allowedFrom && from != relayer) revert InvalidAddress();
```

It then transfers shares from `from` and credits them to `auth.borrower` (passed as `creditTo`):

```
// _depositCollateral (lines 499, 501)
ICTF(ctf).safeTransferFrom(from, address(this), tokenId, amount, "");
// ...
positions[creditTo][tokenId].collateralAmount += amount;
```

The only authorization check is an EIP-712 signature from `auth.borrower`; `auth.allowedFrom` is a field inside that struct and no signature from the address at `allowedFrom` is required.

An attacker sets `auth.allowedFrom` to a victim who has called `setApprovalForAll` on the CTF contract, signs the `LeverageAuth` as `auth.borrower`, and submits it to the relayer. The relayer is an automated system that validates the borrower's signature but does not verify that `allowedFrom` consented; it calls `leverageDeposit` with `from = auth.allowedFrom` and the victim's shares are credited to the attacker's position.

Proof of Concept

Add the following test to `test/PredmartLendingPool.t.sol`:

```
/// @dev PoC for finding high-03:
///      Borrower can set `allowedFrom` to any victim who approved the pool on the
CTF,
///      and relayer can pull the victim&#x27;s ERC-1155 shares into borrower&#x27;s
collateral.
function test_PoC_LeverageDeposit_DrainsApprovedVictimShares() public {
    address victim = makeAddr("victim");
    uint256 victimShares = 4_200e6;
    ctf.mint(victim, TOKEN_ID_YES, victimShares);

    // Victim has previously approved the pool (common UX for deposits).
    vm.prank(victim);
    ctf.setApprovalForAll(address(pool), true);

    uint256 nonce = pool.leverageNonces(borrower);
    uint256 deadline = block.timestamp + 300;
    uint256 maxBorrow = 0; // no borrow needed for the theft; this is a deposit-only
call

    PredmartLendingPool.LeverageAuth memory auth = PredmartLendingPool.LeverageAuth({
        borrower: borrower,
        allowedFrom: victim, // attacker-controlled field inside borrower signature
        tokenId: TOKEN_ID_YES,
        maxBorrow: maxBorrow,
        nonce: nonce,
        deadline: deadline
    });
    bytes memory sig = _signLeverageAuth(borrowerPrivateKey, borrower, victim,
TOKEN_ID_YES, maxBorrow, nonce, deadline);

    assertEq(ctf.balanceOf(victim, TOKEN_ID_YES), victimShares, "Victim shares before
leverage deposit");

    // Relayer pulls shares from victim and credits them to borrower.
    vm.prank(relayer);
    pool.leverageDeposit(auth, sig, victim, relayer, victimShares, 0,
_signPrice(TOKEN_ID_YES, 0.80e18));
```

```
    assertEq(ctf.balanceOf(victim, TOKEN_ID_YES), 0, "Victim shares drained");
    assertEq(ctf.balanceOf(address(pool), TOKEN_ID_YES), victimShares, "Pool received
victim shares");

    {
        (uint256 collateral,,,) = pool.positions(borrower, TOKEN_ID_YES);
        assertEq(collateral, victimShares, "Borrower credited with victim
collateral");
    }

    {
        (uint256 collateral,,,) = pool.positions(victim, TOKEN_ID_YES);
        assertEq(collateral, 0, "Victim shares drained");
    }
}
```

Run the test:

```
forge test --match-test test_PoC_LeverageDeposit_DrainsApprovedVictimShares -vvv
```

Impact

An attacker can steal CTF shares from any address that has granted approval to the contract, credit them as collateral to their own position, and borrow USDC against them, leaving the victim with no shares and no debt compensation.

Recommendation

Require that `auth.allowedFrom == auth.borrower` so borrowers can only deposit from their own address. If the protocol must support depositing from a third-party address (e.g. a Gnosis Safe), collect and verify a separate EIP-712 signature from `auth.allowedFrom` committing to the specific leverage operation, and validate it with `SignatureChecker.isValidSignatureNow()` to support both ECDSA and EIP-1271.

Status

Resolved

Medium

[M-01] PredmartPoolExtension#pullUsdcForLeverage - Operation fee is charged to pool reserves instead of the borrower

Description

In the advance based leverage flow, the `operationFee` is deducted from the pool advance before it leaves the contract, but the resulting debt registered against the borrower excludes the fee. Depositors bear the fee cost instead of the leveraging user.

Vulnerability Details

`pullUsdcForLeverage` deducts `operationFee` from the advance before sending it to the relayer and tracks only the net amount in `pendingAdvances`:

```
// PredmartPoolExtension.sol:674-688
uint256 fee = operationFee;
if (advanceAmount <= fee) revert AdvanceTooSmall();
uint256 netAdvance = advanceAmount - fee;
operationFeePool += fee;
// ...
pendingAdvances[authHash] += netAdvance;
```

At the same time, `leverageBorrowUsed[authHash]` is incremented by the **gross** `advanceAmount` (line 660), consuming the user's full `maxBorrow` budget for that amount.

When `leverageDeposit` later formalises the advance as debt, the `advance` variable is read from `pendingAdvances`, the net figure. When `maxBorrow == advanceAmount`, passing `borrowAmount = advanceAmount` sets `effectiveNewBorrow = fee` and pushes `newTotal` above the budget, reverting. The relayer must pass `borrowAmount = netAdvance` or the call reverts:

```
// PredmartLendingPool.sol:687-703
uint256 advance = pendingAdvances[authHash];          // = advanceAmount - fee
uint256 settled = advance > borrowAmount ? borrowAmount : advance;  // = netAdvance
_advanceOffset = settled;
```

```
uint256 effectiveNewBorrow = borrowAmount > advance ? borrowAmount - advance : 0; //
= 0
// _executeBorrow called with borrowAmount = netAdvance
```

Inside `_executeBorrow`, `offset = netAdvance` suppresses the fee charge (`chargeFee && offset == 0` is false), and `pos.borrowedPrincipal += netAdvance`. The borrower's debt is `netAdvance`, the fee is never included in the debt.

In the non-advance borrow path, `_executeBorrow` sets `toSend = amount - fee`, so the pool sends out less than the recorded debt and retains the fee as a gain. In the advance path the pool sends out exactly what it records as debt, and the `operationFeePool` increment is sourced from existing depositor reserves.

Impact

Every call to `pullUsdcForLeverage` transfers `operationFee` USDC from depositor reserves to the fee pool without charging the borrower.

Recommendation

Consider increasing user debt by the `operationFee` and update all state variables accordingly in `_executeBorrow` when `offset > 0`.

Status

Resolved

[M-02] PredmartLendingPool#_availableCash - Available cash double discounts pending advances

Description

`PredmartLendingPool` subtracts `totalPendingAdvances` from the contract's USDC `balanceOf` when computing `_availableCash()`, even though pending advances have already been transferred out of the pool, causing available liquidity to be understated.

Vulnerability Details

In `pullUsdcForLeverage()`, the pool "advances" USDC to the relayer by incrementing `totalPendingAdvances` / `pendingAdvances[authHash]` and then transferring the advanced USDC out of the pool to `msg.sender` (the relayer). Later, in `leverageDeposit()`, the protocol "settles" the advance by decrementing `totalPendingAdvances` as the advance is offset against a real borrow amount, but no USDC is transferred back into the pool during this settlement.

Despite the fact that the advanced USDC has already left the pool (thereby reducing `IERC20(asset()).balanceOf(address(this))`), `_availableCash()` further subtracts `totalPendingAdvances` from the pool's current USDC balance:

```
uint256 cash = IERC20(asset()).balanceOf(address(this));
cash = cash > totalReserves ? cash - totalReserves : 0;
cash = cash > unsettledRedemptions ? cash - unsettledRedemptions : 0;
cash = cash > operationFeePool ? cash - operationFeePool : 0;
cash = cash > totalPendingAdvances ? cash - totalPendingAdvances : 0; // DOUBLE
DISCOUNT
return cash;
```

The same subtraction is duplicated in `pullUsdcForLeverage()`'s inline liquidity check used to gate new advances.

As a result, any non zero `totalPendingAdvances` causes `_availableCash()` to under report real on hand USDC, since the "pending advance" amount is effectively discounted twice: once by having already reduced the token balance, and again by subtracting `totalPendingAdvances` from that reduced balance.

Impact

Lenders may be incorrectly prevented from withdrawing/redeeming even when the pool still holds sufficient USDC liquidity.

Recommendation

Do not subtract `totalPendingAdvances` from `IERC20(asset()).balanceOf(address(this))` when computing liquidity in `_availableCash()` and inline in `pullUsdcForLeverage()`.

Status

Resolved

Low

[L-01] PredmartLendingPool#setExtension - Extension cannot be rotated when timelock is active

Description

When `timelockDelay` is enabled, `PredmartLendingPool.setExtension()` becomes unusable after the initial extension is set, preventing any future extension rotation through normal governance/admin operations.

Vulnerability Details

The intent described in the NatSpec for `setExtension()` is that updating the extension should be "timelocked when extension is already set." However, the implementation reverts whenever both the extension is already set and the timelock delay is greater than 0.

This creates a permanent lockout once the protocol activates the timelock and has an extension configured. There is no "propose extension change" / "execute extension change after delay" flow, and there is no alternative path in `PredmartPoolExtension` to rotate the extension address.

As a result, the only practical way to change the delegatecall target is to include an extension update inside a UUPS upgrade (e.g., via a new reinitializer setting `extension`), which is operationally heavier than rotating the extension address itself.

Impact

The protocol can be forced into upgrade only extension rotation, reducing operational flexibility to respond quickly to extension level defects and increasing the risk/cost of recovery actions.

Recommendation

Implement a timelocked extension rotation mechanism consistent with other timelocked changes, e.g. add `pendingExtension`/`pendingExtensionExecAfter` with propose/execute/cancel functions in the extension contract.

Status

Resolved

[L-02] PredmartPoolExtension#transferAdmin - Missing AdminTransferred event when timelockDelay is zero

Description

When `timelockDelay` is zero, `transferAdmin` silently reassigns the admin without emitting the `AdminTransferred` event, breaking off-chain observability of ownership changes.

Vulnerability Details

`transferAdmin` takes two branches depending on `timelockDelay`. When the delay is non-zero, the function stages a pending transfer and emits `AdminTransferProposed`. When the delay is zero, it performs an instant assignment (`admin = newAdmin`) with no event emission. `AdminTransferred` is only emitted in `executeTransferAdmin`, which is unreachable when `timelockDelay == 0`.

Impact

Admin ownership changes made while `timelockDelay` is zero are not observable via events, causing off-chain monitors and indexers to hold a stale view of the admin address.

Recommendation

Emit `AdminTransferred` also in the instant transfer case.

Status

Resolved

[L-03] PredmartPoolExtension#activateTimelock - No maximum bound on timelock delay

Description

`activateTimelock` enforces a one-way ratchet that prevents the delay from ever being decreased, but sets no upper bound on the value that can be assigned.

Vulnerability Details

`activateTimelock` accepts any `delay` value greater than or equal to the current `timelockDelay` and writes it directly to storage. All timelocked operations compute their execution window as `block.timestamp + timelockDelay`. If an admin accidentally passes an excessively large value, every subsequent `proposeAddress` call covering oracle, relayer, and upgrade changes will produce an `execAfter` timestamp unreachable in practice. Because the ratchet prevents any downward correction, those critical administrative operations are permanently frozen.

Impact

An accidental assignment of an excessively large timelock delay permanently bricks oracle, relayer, and upgrade change proposals with no recovery path.

Recommendation

Introduce a `MAX_TIMELOCK_DELAY` constant and revert if the provided value exceeds it:

```
uint256 public constant MAX_TIMELOCK_DELAY = 10 days; // Or whatever value is
appropriate for the protocol

function activateTimelock(uint256 delay) external onlyAdmin {
    if (delay < timelockDelay) revert TimelockCannotDecrease();
    if (delay > MAX_TIMELOCK_DELAY) revert TimelockExceedsMaximum();
    timelockDelay = delay;
    emit TimelockActivated(delay);
}
```

Status

Resolved

[L-04] PredmartPoolExtension#pullUsdcForLeverage - Missing frozen token check allows advance to proceed for undepositable collateral

Description

`pullUsdcForLeverage` does not check `frozenTokens[tokenId]` before disbursing USDC to the relayer, while `leverageDeposit`, which must be called afterward to deposit the purchased CTF shares, rejects frozen tokens unconditionally through `_depositCollateral`.

Vulnerability Details

`pullUsdcForLeverage` validates that the market is not resolved but does not check `frozenTokens[auth.tokenId]`. It then transfers funds and sends a pool advance to the relayer. The relayer then uses these funds to buy CTF shares and calls `leverageDeposit`, which calls `_depositCollateral`, which checks `frozenTokens`:

```
// PredmartLendingPool.sol:495
if (frozenTokens[tokenId]) revert TokenFrozen();
```

When `tokenId` is frozen, `leverageDeposit` reverts. The relayer is left holding CTF shares it cannot deposit, the pool's cash balance is short by the advance amount, and `pendingAdvances[authHash]` remains non-zero. The relayer must sell the CTF shares (incurring spread loss), the admin must call `expireAdvance` to fix the accounting, and the relayer must manually return the USDC to the user and/or pool.

Impact

When a token is frozen and a leverage operation is allowed to be initiated, the relayer suffers a USDC loss from the buy/sell spread and the admin must manually expire the pending advance to restore pool accounting.

Recommendation

Add a `frozenTokens` check to `pullUsdcForLeverage` alongside the existing `resolvedMarkets` check.

Status

Resolved

[L-05] PredmartPoolExtension#pullUsdcForLeverage - Missing pause guard allows fund movement during emergency freeze

Description

`pullUsdcForLeverage` lacks a `whenNotPaused` modifier, allowing the relayer to initiate a leverage operation and move funds out of the protocol while the system is paused.

Vulnerability Details

The two-step leverage flow starts with `pullUsdcForLeverage` in `PredmartPoolExtension` and concludes with `leverageDeposit` in `PredmartLendingPool`. `leverageDeposit` is correctly guarded with `whenNotPaused`, but `pullUsdcForLeverage` is not.

When the protocol is paused, the relayer can still call `pullUsdcForLeverage` to transfer funds and record the advance. The follow up call to `leverageDeposit` will revert due to its own pause check, leaving the relayer holding both user and pool funds with no path to complete the operation until the protocol is unpaused.

Impact

While the protocol is paused, the relayer can transfer user and pool funds and mutate borrow accounting state, bypassing the emergency freeze intended to halt borrow operations.

Recommendation

Add `whenNotPaused` to `pullUsdcForLeverage`.

Status

Resolved

[L-06] PredmartLendingPool#depositCollateral - Contract depositor cannot withdraw collateral

Description

Collateral deposits are credited to `msg.sender`, but collateral withdrawals require an ECDSA signature from the borrower, which is incompatible with smart contract accounts.

Vulnerability Details

`depositCollateral()` credits the collateral position to `msg.sender`:

```
function depositCollateral(uint256 tokenId, uint256 amount) external nonReentrant
whenNotPaused {
    _depositCollateral(msg.sender, msg.sender, tokenId, amount);
}
```

Collateral withdrawals are performed via `withdrawViaRelay()`, which verifies the borrower via `ECDSA.recover(...) == intent.borrower`:

```
address signer = ECDSA.recover(_hashTypedDataV4(structHash), intentSignature);
if (signer != intent.borrower) revert InvalidIntentSignature();
```

This signature validation supports EOAs only and does not support EIP-1271 contract signatures. As a result, if a smart contract deposits collateral (so the position is recorded under the contract address as `borrower`), it cannot produce a valid ECDSA signature "from itself" to authorize `withdrawViaRelay()`, and the protocol provides no alternative withdrawal path for the borrower to exit the position.

Impact

Collateral deposited by a smart contract address can become stuck and unwithdrawable.

Recommendation

Use an EIP-1271 compatible signature check (e.g., OpenZeppelin `SignatureChecker.isValidSignatureNow()`) for `withdrawViaRelay()` and others functions

of the protocol that require borrower signature, so contract accounts can withdraw without relying on ECDSA recovery.

Status

Resolved

QA

[Q-01] PredmartLendingPool#supportsInterface - No-op override with misleading conflict comment

Description

The `supportsInterface` override exists to resolve a claimed conflict between `ERC1155Holder` and `ERC4626Upgradeable`, but `ERC4626Upgradeable` does not define `supportsInterface`, so no conflict exists and the override adds no behaviour.

Vulnerability Details

Of the five base contracts — `ERC4626Upgradeable`, `UUPSUpgradeable`, `EIP712Upgradeable`, `ReentrancyGuard`, and `ERC1155Holder` — only `ERC1155Holder` defines `supportsInterface`.

Solidity requires `override(A, B)` syntax when two or more bases both define a virtual function; a single `override` is only legal when exactly one parent provides the implementation. The plain `override` here confirms that `ERC4626Upgradeable` plays no part in the resolution. The override body delegates unconditionally to `super.supportsInterface(interfaceId)`, which resolves to `ERC1155Holder.supportsInterface`.

Impact

The override is a no-op and its comment misrepresents the inheritance hierarchy and falsely asserts a conflict.

Recommendation

Remove the `supportsInterface` override.

Status

Resolved

[Q-02] PredmartLendingPool#_verifyRelaySignature - Unused relay signature verifier

Description

`PredmartLendingPool` defines `_verifyRelaySignature()` to validate relayed EIP-712 intents, but no function calls it, so it is dead code and duplicates checks that are implemented inline elsewhere.

Vulnerability Details

`_verifyRelaySignature()` validates the relayer, deadline, and signature of an EIP-712 intent. However, `withdrawViaRelay()` and `borrowViaRelay()` re-implements similar relayer/deadline/signature validation logic directly instead of calling `_verifyRelaySignature()`.

`_verifyRelaySignature()` has no references beyond its own definition, meaning its presence does not affect execution and can easily diverge from the actual checks enforced in the public relay entry points.

Impact

Dead/duplicated verification code increases maintenance.

Recommendation

Either remove `_verifyRelaySignature()` entirely, or refactor relayed entry points like `withdrawViaRelay()` and `borrowViaRelay()` to consistently use it so there is a single canonical implementation of relay signature verification.

Status

Resolved

Centralisation

The Predmart project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.

Conclusion

After Hashlock's analysis, the Predmart project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au
Contact: info@hashlock.com.au